



Módulo 3 · Testing, Debugging y Documentación con IA

Integración, Datos de Prueba y Buenas Prácticas con IA

Ejercicio de Clase · Ejercicio 2 de 2 · Bloques 1.4★— 1.5 — 1.6 — 1.7

Nombre del alumno/a	Angel Echenique
Fecha	22/03/2026
🔗 Enlace GitHub del repositorio	

📋 **Instrucciones:** Responde con lo que vimos en clase. Puedes consultar tus apuntes. ¡Solo demostrar que entendiste, no hay que escribir código!

BLOQUE 1 — Estructura del prompt — Aplicación avanzada (1.4★)

En este bloque aplicamos lo visto sobre la estructura del prompt con contexto real.

Completa la tabla con los **6 elementos** del prompt profesional,

añadiendo ahora un ejemplo concreto para cada uno:

Nº	Elemento	¿Para qué sirve?	Ejemplo
1	Función o módulo	Indica qué quieres testear	procesar_pedido(pedido)
2	Framework	Define la herramienta	"Usa pytest"
3	Casos edge	Añade límites importantes	pedido vacío o precio 0
4	Mocks	Simular dependencias	mock de base de datos
5	Formato	Cómo quieres la respuesta	tests con estructura AAA
6	Explicación	Entender los tests	comentario en cada test



1.2 — El impacto del contexto de negocio

Vimos que añadir contexto de negocio transforma los tests. Completa la columna derecha con lo que cambiaría si la función trabaja en un **sistema financiero**:

Aspecto del test	Sin contexto de negocio	Con contexto financiero
Tipo de datos numéricos	float	Decimal preciso.
Precisión decimal	15 decimales	2 decimales dinero.
Valores negativos	Permitidos	No permitidos
Logging de operaciones	No	Obligatorio
Número de tests generados	~15	25+ con validaciones



1.3 — Construye el prompt correcto

Tu compañero/a necesita una función que procese pedidos de una tienda online. Ha escrito este prompt:

"Hazme una función para pedidos."

Reescribe el prompt correctamente aplicando el nivel que corresponde. Incluye **inputs**, **outputs** y al menos **un caso límite**:

Crea una función en Python llamada `procesar_pedido(pedido)` que:

INPUT:

- pedido: diccionario con productos, precios y cantidades

OUTPUT:

- total del pedido

REQUISITOS:

- Validar que los precios sean positivos
- Calcular total correctamente
- Manejar pedidos vacíos

CASO LÍMITE:

- pedido vacío →devolver 0 o lanzar error

Incluye validaciones y manejo de errores.

BLOQUE 2 — Pruebas de integración con IA (1.5)

Las pruebas de integración verifican que varios componentes funcionen correctamente juntos. Completa la tabla comparativa:

Característica	Unit Test	Integration Test
¿Qué prueba?	Una función/clase aislada	Sistema completo
Velocidad	Milisegundos	Mas lento
Aislamiento	Sí (con mocks)	no
Dependencias	Ninguna	BD, API
Confianza	Media (componente funciona)	Alta
Número de tests	Muchos (cientos/miles)	Pocos
Fallo indica	Función específica rota	Interacción rota
Ejecución	En cada commit	Antes de deploy

2.2 — ¿Qué puede generar la IA en pruebas de integración?

Para cada tipo de test de integración, escribe un **ejemplo de prompt** que usarías para pedirselo a la IA:

Qué genera la IA	Descripción	Escribe un ejemplo de prompt para pedirlo
Tests para endpoints REST	La IA genera tests completos para rutas POST, GET, PUT, DELETE...	Creame tests de integración con pytest para el endpoint POST /login incluyendo validación de respuesta y status code.
Mocks de base de datos	Simula consultas, respuestas de repositorios y ORMs	Genera tests con mocks de base de datos para simular guardado de usuarios sin usar una BD real.
Flujos completos	Construye escenarios: registro →login →acción →logout	Genera un test de integración que simule registro →login →acción →logout.
Tests con supertest / pytest / JUnit	Frameworks específicos por lenguaje para integración	Genera tests de integración usando pytest para una API REST completa.

2.3 — Flujo completo

Describe los pasos de un **flujo completo de integración** que la IA podría generar para un sistema de registro de usuarios (registro →login →acción →logout). Incluye qué se valida en cada paso:

Registro envia datos y se valida que usuario se guarda, login se autentica y se obtiene token, accion lo que hace el usuario y se valida permiso por ultimo loguot cerramos sesion token invalido.

BLOQUE 3 — IA generando datos de prueba (1.6)

La IA puede generar **5 tipos de datos** de prueba. Clasifica cada ejemplo usando: **V** (Válido), **I** (Inválido), **L** (Límite), **A** (Aleatorio), **M** (Masivo):

Tipo (V/I/L/A/ M)	Ejemplo de dato	¿Por qué es de ese tipo?
V	email: "usuario@ejemplo.com", edad: 25, país: "España"	Es un dato válido porque cumple las reglas del sistema (formato email correcto, dominio válido, etc.). Se usa para verificar que el sistema funciona correctamente en condiciones normales (happy path)
I	email: "usuariosindominio", edad: -5, contraseña: "ab"	Es inválido porque rompe múltiples reglas: el email no cumple formato y la edad es negativa (fuera de dominio lógico). Sirve para probar validaciones y manejo de errores
L	contraseña de exactamente 8 caracteres, edad = 0 o edad = 120	Es un valor límite porque está en los extremos del rango permitido (mínimo y máximo). Aquí se busca detectar errores típicos como off-by-one ($\geq$ vs $>$).
M	10.000 usuarios generados con datos coherentes para pruebas de carga	Es masivo porque implica gran volumen de datos. Se usa para pruebas de rendimiento, carga o estrés del sistema
A	nombres generados con valores únicos, emails con tokens aleatorios	Es aleatorio porque no sigue un patrón específico. Se usa para encontrar bugs inesperados (fuzz testing, robustez).
I	email: "", campo obligatorio sin rellenar, tipo string en campo numérico	Es inválido porque viola reglas de obligatoriedad (required fields). Se comprueba que el sistema no acepte datos incompletos.



L	pedidos con precios entre 0.01 y 9999.99, nombres de 2 a 50 caracteres	Es límite porque define el rango permitido. Los valores cercanos a 0.01 y 9999 son críticos para detectar errores en validaciones numéricas o precisión.
---	--	---

3.2 — Construye el prompt para datos de prueba

Escribe el prompt que usarías para pedirle a la IA que genere datos de prueba para un sistema de registro de usuarios. Incluye al menos **3 tipos de datos distintos** y especifica el framework:

Actúa como un ingeniero de QA especializado en testing automatizado con pytest.

Genera datos de prueba para un sistema de registro de usuarios con los siguientes campos:

- email (string, formato válido de correo).
- password (mínimo 8 caracteres, debe incluir números y letras).
- edad (entero entre 0 y 120).

Necesito que generes los datos organizados en categorías:

1. Casos válidos (V): datos completamente correctos.
2. Casos inválidos (I): errores en formato, valores negativos, campos vacíos y lo que veas necesarios..
3. Casos límite (L): valores en los extremos del rango permitido.

Devuelve la respuesta en formato compatible con pytest parametrizado, por ejemplo:

@pytest.mark.parametrize("email, password, edad, esperado", [...]).

Incluye al menos:

- 3 casos válidos.
- 3 inválidos.
- 3 de límite.

Añademe una breve explicación por cada caso indicando qué se está probando con docstring.

BLOQUE 4 — Buenas prácticas al usar IA para testing (1.7)

En clase vimos **6 buenas prácticas** que debemos aplicar siempre. Escríbelas y explica brevemente para qué sirve cada una:

Nº	Nombre de la buena práctica	¿Para qué sirve? ¿Qué problema evita?
1	Revisar y validar los tests generados	Sirve para asegurarnos que los tests tienen sentido lógico. Evita falsos positivos/negativos y tests inútiles.
2	No confiar ciegamente en la IA	La IA puede equivocarse o asumir cosas incorrectas. Evitamos errores silenciosos en producción.
3	Especificar bien el contexto en el prompt	Cuanto más claro el input (reglas, validaciones, tipos), mejor output. Evitamos tests genéricos o irrelevantes.
4	Combinar IA + criterio humano	La IA ayuda, pero el QA define qué probar. Evita depender totalmente de la IA y perder calidad.
5	Cubrir edge cases manualmente	La IA suele centrarse en casos comunes. Evita dejar sin probar escenarios críticos (null, límites raros, tipos incorrectos).
6	Iterar y refinar los prompts	Permite mejorar progresivamente los resultados. Evita outputs pobres por prompts vagos o incompletos.

4.2 — Buena práctica o error común

Marca **BP** (Buena Práctica) o **EC** (Error Común). Para los errores, escribe también cómo se corregiría:

BP / EC	Afirmación	Corrección (solo si es EC)
BP	Revisar cada test generado para comprobar que tiene sentido y no hay falsos positivos.	
EC	Usar los tests de la IA directamente sin ejecutarlos, para ahorrar tiempo.	Siempre ejecutar y validar los tests antes de usarlos , verificando que fallan/pasan correctamente.
BP	Documentar en el código qué tests fueron generados por IA y cuándo fueron revisados.	
EC	Asumir que los mocks generados por la IA representan exactamente el comportamiento real de la API.	Validar mocks contra la API real o documentación , y ajustarlos manualmente si es necesario.
BP	Añadir manualmente casos edge que la IA no haya incluido, como valores nulos o tipos inesperados.	
EC	Pedir a la IA todos los tests del sistema en un único prompt para ser más eficiente.	Dividir en prompts pequeños y específicos (por módulo o funcionalidad).
BP	Mantener el estilo AAA (Arrange-Act-Assert) de forma coherente en todos los tests del proyecto.	

BLOQUE 5 — Caso real — Síntesis

▢ **Caso real:** Un equipo está desarrollando un sistema de pagos online. El jefe de proyecto propone: "Usemos la IA para generar todos los tests de integración del módulo de pagos y los desplegamos directamente sin revisarlos, para cumplir el plazo."

5.1 — ¿Qué riesgos tiene esta decisión?

Menciona al menos **3 riesgos concretos** en un sistema de pagos real, relacionándolos con lo visto en los 4 bloques:

A simple vista que los pagos efectuados no sean validos por errores de la IA al escribir el codigo lo cual fallaria en produccion y si no se dan cuenta a tiempo puede que hasta se permita pagos negativos.

5.2 — ¿Cómo lo harías tú?

Describe cómo usarías la IA de forma responsable. Menciona qué harías antes, durante y después:

Momento	Qué harías (pasos concretos)
Antes de pedir los tests a la IA	➤ Definir requisitos del sistema. ➤ Identificar casos edge.
Al escribir el prompt para la IA	➤ Escribir prompt claro. ➤ Incluir contexto de negocio.
Después de recibir los tests generados	➤ Revisar todos los tests. ➤ Ejecutarlos. ➤ Mejorarlos manualmente.



Antes de desplegar a producción	➤ Validar con datos reales. ➤ Revisar integración completa. ➤ Asegurar cobertura.
--	---